

Modeling Continuous Data

Week 10

Natalie Agus

Learning Objectives

- Write **cost** function of linear regression
- Implement **Gradient Descent algorithm** for **optimisation**
- **Train** linear regression model using gradient descent
- **Evaluate** linear regression model using r^2 and mean-squared-error
- **Plot cost function** over iteration time
- **Plot linear regression**
- **Transform** data for polynomial model

House prices → continuous data
2000 - 2024

IV →

④ → LUM self-supervised.
③ → Reinforcement

① supervised learning → labelled dataset
lin reg. → cont
log reg. → classification

② unsupervised learning → pattern / dist
↓ → unlabelled datasets
K means

100 → Train set 70%
↓ ONLY Test set 30% (30).

70% → actual train set 80%
mock test validation set 20%
to see (24) if we need improvements

Linear Regression

- ^{features} Independent vs ^{target} dependent variable
- Try to model relationship between the two: e.g. linear relationship

$$y = mx + c$$

- Goal: given arbitrary independent variable, **predict** dependent variable

①

Hypothesis

A proposed explanation for a phenomenon

- A hypothesis models the relationship between dependent and independent variables. Also known as **model**
- It contains **parameters**
- We need to **train** our model (hypothesis) to figure out the **value** of the parameters
- These parameters later are used for **prediction**

✓ actual value

$$y = \beta_0 + \beta_1 x$$

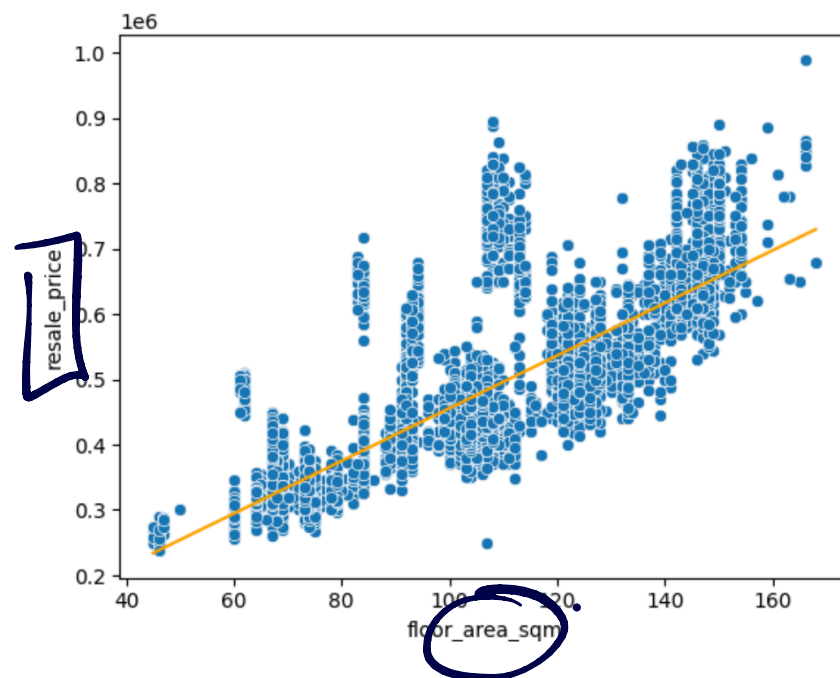
$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 \hat{x}$$

predicted value

new feature

Hypothesis

A proposed explanation for a phenomenon

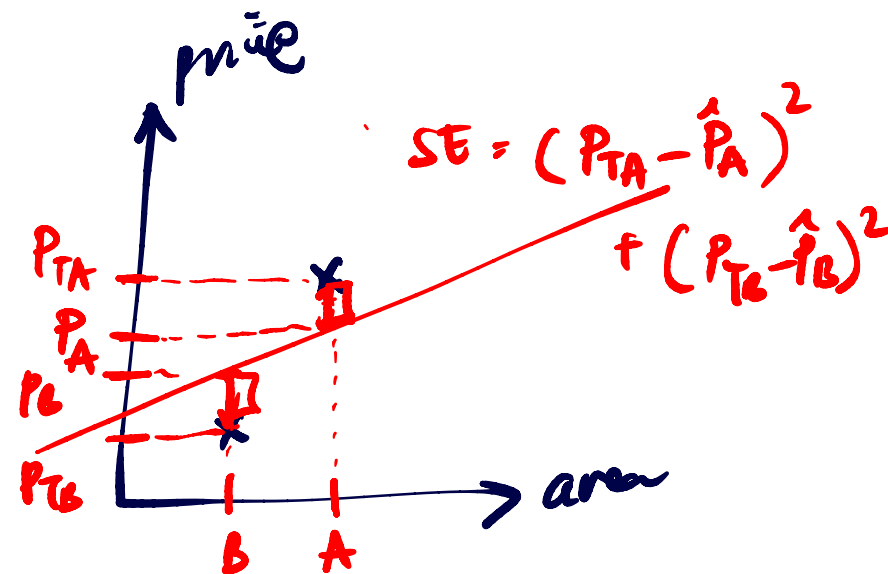


$$y = \beta_0 + \beta_1 x$$

$$\hat{\beta}_0 = 52643$$

$$\hat{\beta}_1 = 4030$$

- ① plot
- ② choose a model
- ③ If ~~linear~~ linear
 $\hat{y} = \beta_0 + \beta_1 x$
- ④ Find an error metric.



Cost Function

- We initially only have a **hypothesis** with unknown **parameters**
- We need to **train** that model to figure out the optimum **parameters**
- How do we know if the training goes well or not?
- Using **cost function** $RSS = \sum_{i=1}^m (\hat{y}(x^i) - y^i)^2$ Trainset
pred true
- It measures the **difference** between the model's predictions and the actual values
- Many types of cost function: Residual Sum Square, Mean Squared Error, Mean Absolute Error, Huber Loss, KL Divergence, Hinge Loss, Poison Loss, etc

Standard steps

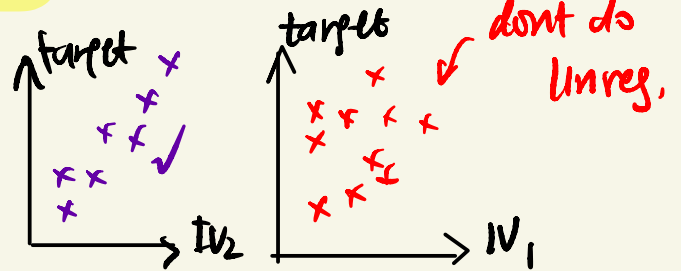
Step 0: gather dataset, clean up $\rightarrow$ normalization $\rightarrow$ estimate null values

Step 1: plot, and guess relationship b/w IV & target

Step 2: come up w/ the hypothesis (model)

$$e.g. \rightarrow \hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \text{ (linear)}$$

$$\rightarrow \hat{y} = \beta_0 + \beta_1 x_1^2 + \beta_2 x_1 \text{ (quadratic)}$$



Step 3: come up w/ cost function $\rightarrow$ RSS

$\rightarrow$ MSE

$\downarrow$ MAE

$$J = \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$J = \frac{1}{2m} \sum_{i=1}^m |\hat{y}_i - y_i|$$

Step 4: optimize your params (train) $\rightarrow$ find β s that will minimize J (error)

$$J(\beta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i(\beta) - y_i)^2$$

$\hat{y}_i(\beta)$ $\rightarrow$ predicted
 y_i $\rightarrow$ actual

$\downarrow$
* we use gradient descent

① guess init value of β

② compute the rate of change of J w/ the params.

$$\frac{\partial J}{\partial \beta} > 0 \text{ or } < 0 ?$$

$$\beta_0 = 5 \rightarrow \frac{\partial J}{\partial \beta_0} = -3 \rightarrow \text{increase } \beta_0$$

③ update β :

$$\beta_{new} = \beta_{old} - \frac{\partial J}{\partial \beta_{old}} * \alpha$$

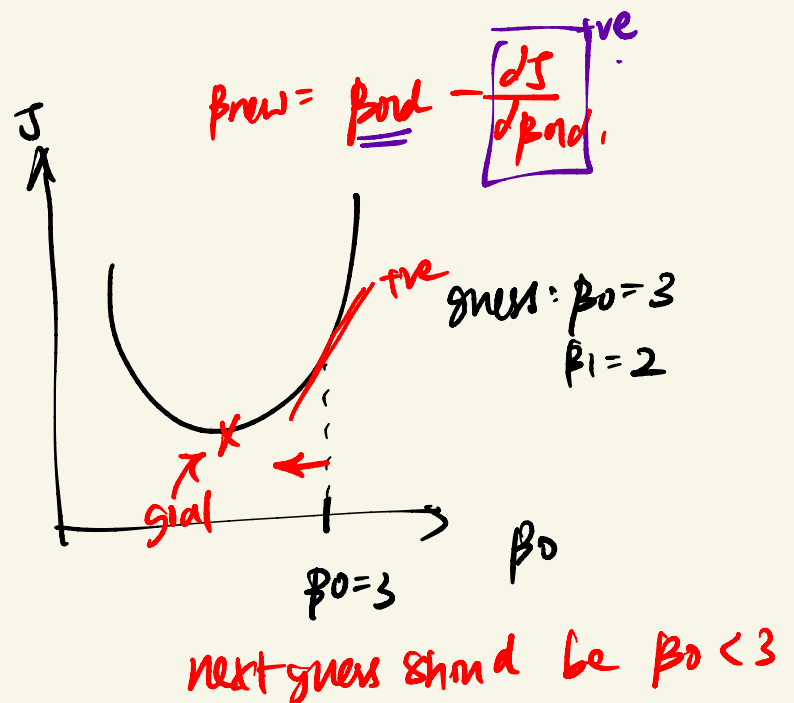
$$\beta_1 = 5 \rightarrow \frac{\partial J}{\partial \beta_1} = 3 \rightarrow ? \text{ decrease } \beta_1$$

repeat until J is low

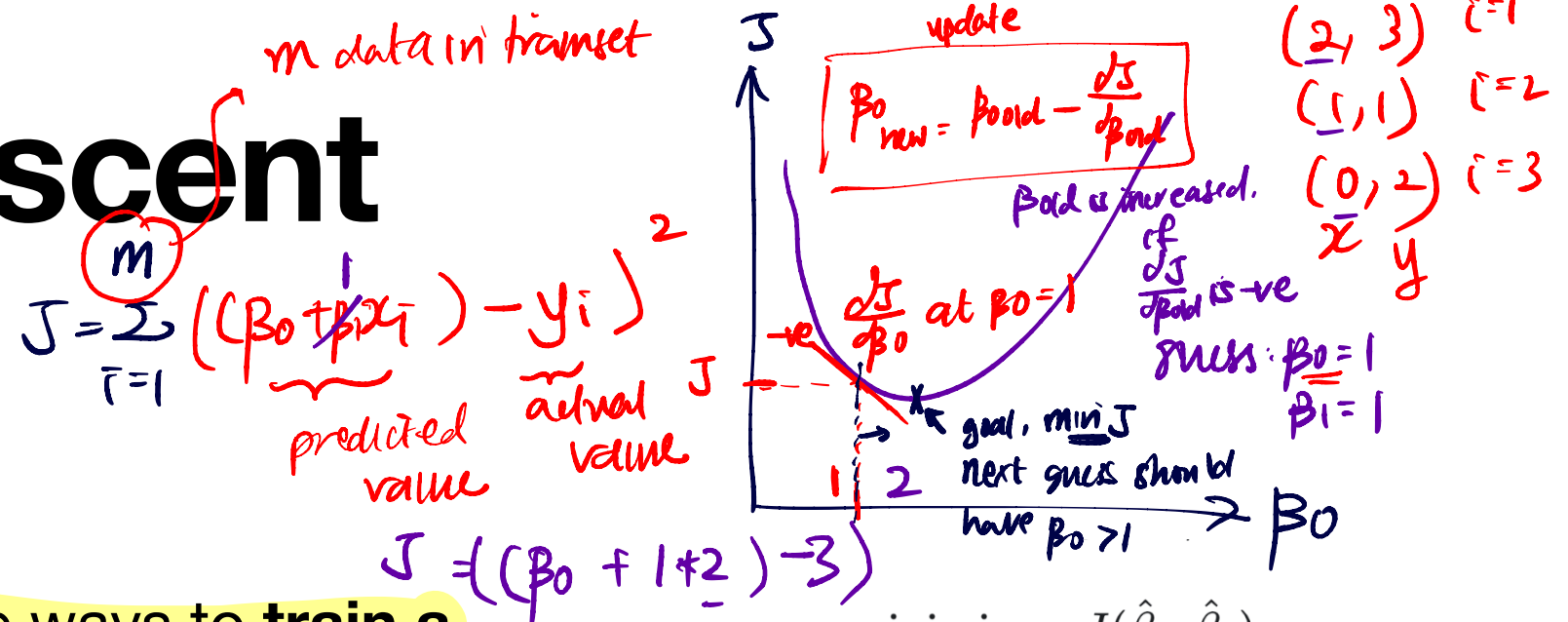
steps : use trained β s on Validation set
↳ check if J is still satisfactory. $\rightarrow$ yes goto 6
no, repeat (2)

step 6 : test performance on test set.

β_{new} is reduced if $\frac{dJ}{d\beta_{\text{old}}}$ is +ve



Gradient Descent



- Gradient decent is one of the ways to **train a model** and find **optimum** parameters.

- Other ways: Stochastic GD, simulated annealing, Newton's method, Bayesian optimisation, genetic algorithm, etc

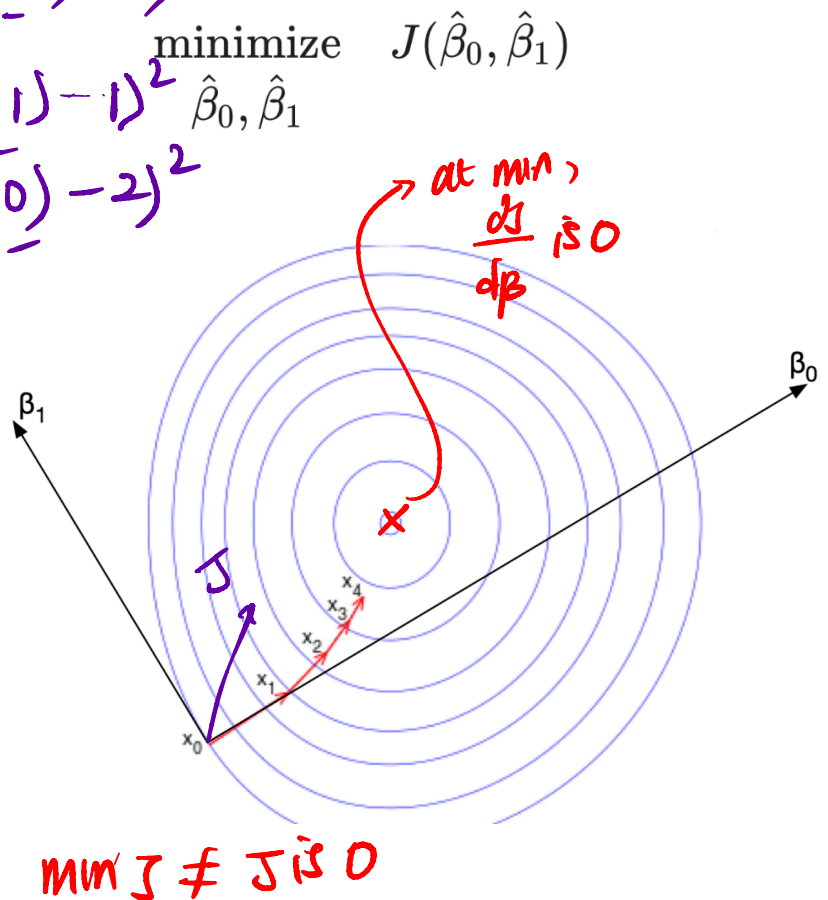
- Goal:** find parameters that **minimise cost function**

- Start:** with initial guess of all parameters

- Then, **compute** the rate of error (gradient of the cost function)

- Next, **update** the parameters

- Repeat**



$$\hat{\beta}_j = \hat{\beta}_j - \alpha \frac{\partial}{\partial \hat{\beta}_j} J(\hat{\beta}_0, \hat{\beta}_1)$$

Gradient Descent

$$\hat{\beta}_0 = \hat{\beta}_0 - \alpha \frac{\partial}{\partial \hat{\beta}_0} J$$

$$\hat{\beta}_1 = \hat{\beta}_1 - \alpha \frac{\partial}{\partial \hat{\beta}_1} J$$

$$J(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}(x^i) - y^i)^2 \quad \text{MSE}$$

$$\frac{\partial}{\partial \hat{\beta}_j} J(\hat{\beta}_0, \hat{\beta}_1) = \frac{\partial}{\partial \hat{\beta}_j} \frac{1}{2m} \sum_{i=1}^m (\hat{y}(x^i) - y^i)^2$$

$$y = \beta_0 + \beta_1 x$$

Update function:

$$\hat{\beta}_0 = \hat{\beta}_0 - \alpha \frac{1}{m} \sum_{i=1}^m (\hat{y}(x^i) - y^i) \quad \checkmark$$

$$\hat{\beta}_1 = \hat{\beta}_1 - \alpha \frac{1}{m} \sum_{i=1}^m (\hat{y}(x^i) - y^i) x^i$$

$$J = \frac{1}{2m} \sum_{i=1}^m ((\beta_0 + \beta_1 x^i) - y^i)^2$$

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{2m} \sum_{i=1}^m ((\beta_0 + \beta_1 x^i) - y^i) \frac{\partial}{\partial \beta_0} ((\beta_0 + \beta_1 x^i) - y^i)$$

Matrix Representation of Hypothesis

- Representing the **hypothesis** as matrices allow us to use matrix operations to clean the data & train the model, **easier** to implement as a program
- Use **NumPy** matrix operations for ease of computation

feature matrix

$$\mathbf{X} = \begin{bmatrix} 1 & x^1 \\ 1 & x^2 \\ \vdots & \vdots \\ 1 & x^m \end{bmatrix}$$

datapts

params

mult

$\hat{\mathbf{b}} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{bmatrix}$

mult

$$\begin{matrix} 1 \times \beta_0 + x^1 \times \beta_1 \rightarrow \hat{y}^1 \\ \vdots \\ 1 \times \beta_0 + x^m \times \beta_1 \rightarrow \hat{y}^m \end{matrix} \rightarrow \begin{bmatrix} \hat{y}^1 \\ \vdots \\ \hat{y}^m \end{bmatrix}$$

all pred values.

$$\hat{\mathbf{y}} = \mathbf{X} \times \hat{\mathbf{b}}$$

1st feat of the 1st data

2nd feat of 1st data

1st data

data index

feat idx

pred target

506 datapts.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \rightarrow \mathbf{X} = \begin{bmatrix} 1 & x_1^1 & x_2^1 \\ \vdots & \vdots & \vdots \\ 1 & x_1^m & x_2^m \end{bmatrix}$$

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

Matrix Representation of Cost Function

$$\hat{y} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} \quad y = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad m=3$$

$$J = (3-1)^2 + (2-1)^2 + (1-1)^2$$

$$\hat{y} - y = \begin{bmatrix} 3-1 \\ 2-1 \\ 1-1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} \rightarrow \text{if I want to sq each element \& sum, we do: } (\hat{y}-y)^T \times (\hat{y}-y)$$

$$J(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}(x^i) - y^i)^2$$

$$J(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}(x^i) - y^i) \times (\hat{y}(x^i) - y^i)$$

$$J(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{2m} (\hat{y} - y)^T (\hat{y} - y)$$

- Representing the **cost function** as matrices allow us to use matrix operations to clean the data & train the model, **easier to implement**

- Use NumPy matrix operations for ease of computation

- Note: **MSE** is used

$$y \Rightarrow \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} \xrightarrow{\text{sq}} \begin{bmatrix} 1 \\ 4 \\ 16 \end{bmatrix} \xrightarrow{\text{sum}} 21 \quad \vec{y}^2 \xrightarrow{\text{sum}} y^T \times y \quad [124] \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} = [1^2 + 2^2 + 4^2] = 21$$

scalar = 21 can remove the list in py.

Matrix Representation of Gradient Descent Update Function

$d = \# \text{ params}$
 $m = \# \text{ of data pts}$

$$\hat{\beta}_0 = \hat{\beta}_0 - \alpha \frac{1}{m} \sum_{i=1}^m (\hat{y}(x^i) - y^i)$$

$$\hat{\beta}_1 = \hat{\beta}_1 - \alpha \frac{1}{m} \sum_{i=1}^m (\hat{y}(x^i) - y^i) x^i$$

$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \xrightarrow{\text{col vect}} \beta = \begin{bmatrix} [\beta_0] & [\beta_1] \end{bmatrix}$ (2 rows 1 col)

$\beta^T = [\beta_0 \ \beta_1]$ (1 row 2 col)

$\mathbf{X} = \begin{bmatrix} 1 & x^1 \\ 1 & x^2 \\ \dots & \dots \\ 1 & x^m \end{bmatrix}$ (m rows 2 col)

$\beta^T \mathbf{X} = \text{np.reshape}(d, -1)$ (col auto at rows)

$$\hat{\mathbf{b}} = \hat{\mathbf{b}} - \alpha \frac{1}{m} \mathbf{X}^T \times (\hat{\mathbf{y}} - \mathbf{y})$$

$$\hat{\mathbf{b}} = \hat{\mathbf{b}} - \alpha \frac{1}{m} \mathbf{X}^T \times (\mathbf{X} \times \hat{\mathbf{b}} - \mathbf{y})$$

$\beta \text{ vector: } d \text{ by } 1$

$\mathbf{X}^T$ (m by d)

$\mathbf{X}$ (m by d)

$\hat{\mathbf{b}}$ (d by 1)

$\mathbf{y}$ (m by 1)

$\mathbf{X} \times \hat{\mathbf{b}}$ (m by 1)

$\mathbf{X} \times \hat{\mathbf{b}} - \mathbf{y}$ (m by 1)

$$\mathbf{X}^T = \begin{bmatrix} 1 & 1 & \dots & 1 \\ x^1 & x^2 & \dots & x^m \end{bmatrix}$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \rightarrow \text{param vector}$$

$$\mathbf{y} = \text{target vector}$$

$$\hat{\mathbf{y}} = \text{pred vector}$$

$$\mathbf{y} = \text{target}$$

Metrics

- Training data set is used to train (build) the hypothesis (model)
- Test data set is used to **evaluate** the model: we compute metrics to determine whether the model is good/bad

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^i - \hat{y}^i)^2$$

$$r^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Multiple Linear Regression

- Model the relationship between a **dependent** variable and **multiple** (no longer single) **independent** variable
- SLR: $\hat{y}(x) = \hat{\beta}_0 + \hat{\beta}_1 x$
- MLR: $\hat{y}(x) = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_n x_n$

Multiple Linear Regression: Matrix Expression

$$\hat{y}(x^1) = \hat{\beta}_0 + \hat{\beta}_1 x_1^1 + \hat{\beta}_2 x_2^1 + \dots + \hat{\beta}_n x_n^1$$

$$\hat{y}(x^2) = \hat{\beta}_0 + \hat{\beta}_1 x_1^2 + \hat{\beta}_2 x_2^2 + \dots + \hat{\beta}_n x_n^2$$

...

$$\hat{y}(x^m) = \hat{\beta}_0 + \hat{\beta}_1 x_1^m + \hat{\beta}_2 x_2^m + \dots + \hat{\beta}_n x_n^m$$

$$\hat{\mathbf{y}} = \mathbf{X} \times \hat{\mathbf{b}}$$

Same as SLR

$$\hat{\mathbf{b}} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \\ \vdots \\ \hat{\beta}_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^1 & \dots & x_n^1 \\ 1 & x_1^2 & \dots & x_n^2 \\ \dots & \dots & \dots & \dots \\ 1 & x_1^m & \dots & x_n^m \end{bmatrix} \in \mathbb{R}^{m \times (n+1)}$$

MLR Cost Function: Matrix Expression

$$J(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{2m}(\hat{\mathbf{y}} - \mathbf{y})^T \times (\hat{\mathbf{y}} - \mathbf{y})$$

Same as SLR

MLR Gradient Descent Update Function: Matrix Expression

$$\hat{\mathbf{b}} = \hat{\mathbf{b}} - \alpha \frac{1}{m} \mathbf{X}^T \times (\mathbf{X} \times \hat{\mathbf{b}} - \mathbf{y})$$

Same as SLR

About R²

- Sometimes it can be **misleading**: R² always increases with more features as it measures the **proportion** of variance explained by the variance and adding features generally **reduces** the RSS, making R² higher
 - R² is good for simple models, easier to interpret
- A high R² does **not** necessarily mean a better model—it might just be overfitting.
- Adjusted R² penalise the inclusion of irrelevant features

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - k - 1} \right)$$

About R^2 vs Adjusted R^2

- Some nuances
- With $R^2 = N\%$, you can say "N% of the **variation** in the application is **explained** by our predictors"
- With Adjusted $R^2 = N$, you must say "After **accounting** for the number of predictors and **penalizing** any unnecessary ones, the model explains around N% variance in the application"

Can R^2 be negative?

- Usually **no**, but if your model predicts **worse** than the average then **yes**
- If your model fits the data worse than a horizontal mean line, that is: $SS_{res} > SS_{tot}$, then R^2 becomes negative.

$$r^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Polynomial Model

Transform features into polynomial terms

- Plot first see the **relationship** between independent and dependent variable

$$x_1 = x$$

$$x_2 = x^2$$

- Transform the independent variable as necessary, e.g: **quadratic hypothesis**

$$\hat{y}(x) = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2$$

- Note:** you **don't always need** the lower order terms if you are confident that only the higher order has predictive power. *Otherwise, it won't "hurt" either, as the params can be near zero if the data doesn't have meaningful relationship with lower order terms*

$$\mathbf{X} = \begin{bmatrix} 1 & x^{(1)} & (x^2)^{(1)} \\ 1 & x^{(2)} & (x^2)^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x^{(m)} & (x^2)^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times 3}$$

Polynomial Model

Considerations

- **Overfitting:** higher degree polynomials can overfit data, capturing **noise** instead of underlying pattern
- **Feature scaling:** polynomial terms can result in large differences in magnitude, requiring normalization or standardization **before** polynomial transformation
- **Application:** modeling non-linear behaviors like trajectories, forces, market trends

Learning Objectives

- Write **cost** function of linear regression
- Implement **Gradient Descent algorithm** for **optimisation**
- **Train** linear regression model using gradient descent
- **Evaluate** linear regression model using r^2 and mean-squared-error
- **Plot cost function** over iteration time
- **Plot linear regression**
- **Transform** data for polynomial model